

Machine-Washed: Prepping Data for AI the Azure Way

Purpose: In many business settings, inconsistent or poorly formatted data creates significant challenges. Data files might contain missing values, mislabeled columns, or incompatible formats, which makes them difficult to work with and slows down analysis. This can lead to delays in reporting, reduced confidence in insights, and increased operational overhead. Manually correcting these issues can be time-consuming, repetitive, and prone to human error, especially when data is collected from multiple sources or handled by different teams. For this project, we set out to design a more efficient solution using Microsoft Azure. Our goal was to create an automated system that could take raw data files, apply cleaning rules, and output consistent, usable datasets without manual intervention. We aimed to reduce time spent on repetitive tasks while improving the overall reliability and quality of the data being used.

Methodology: We began by converting our original Excel dataset into a CSV file, since this format is easier for Azure Data Factory to process. CSV files are lightweight, universally compatible, and ideal for use in automated workflows. Once the file was prepared, we created an Azure Storage Account to host our project's resources. Within the storage account, we set up two separate Blob Storage containers. The first container, labeled "rawdata," was designated for unprocessed input files, while the second container, labeled "cleandata," was intended to hold the cleaned, transformed output. These containers function like cloud-based folders, enabling organized storage and easy access from other Azure services.

To automate the cleaning process, we used Azure Data Factory to build a data pipeline. Within this pipeline, we created a Data Flow that defined each step of the transformation

process. The first step involved establishing linked services, which are secure connections between Azure Data Factory and other Azure components, in this case, our Blob Storage account. These connections ensure that the pipeline can securely access both the input and output locations.

After configuring the linked services, we created two datasets. One represented the source file located in the "rawdata" container, and the other represented the cleaned output file path in the "cleandata" container. These datasets acted as references within the pipeline, allowing the transformations to be applied to the correct files.

The heart of the Data Flow involved a series of transformations designed to correct formatting issues and standardize the data. One key transformation addressed the "MemberID" column, which had been incorrectly stored as a date value. To resolve this, we used a Derived Column transformation that reformatted the date into a string using the "MM/dd/yyyy" format, then converted that string into an integer using a built-in expression. This ensured the ID could be treated as a true numeric value in future processing. A similar correction was made for the "DOB" column, which began as a numeric value in the "yyyymmdd" format. Using another Derived Column step, we converted it into a valid date type that could be properly interpreted by downstream applications.

In addition to fixing data types, we could have used Select and Filter transformations to clean the structure of the dataset. The Select transformation would have allowed us to rename columns for consistency, remove unnecessary fields, and reorganize the schema as needed. The Filter transformation could have been used to exclude rows that did not meet specific criteria,

for example, filtering out records that were missing essential values or did not match expected conditions. However, we did not need to utilize these transformations for our data set.

Once all transformations were configured, we used the Debug feature in Azure Data Factory Studio to test the pipeline. This step allowed us to validate the logic and ensure everything worked as intended without fully committing to a run. After confirming that the transformations were successful, we published the pipeline and triggered it. The result was a new, cleaned file that was automatically stored in the "cleandata" container, ready for use.

Result Analysis: The final system worked efficiently and successfully cleaned the dataset with minimal user input after setup. Once the pipeline was in place, we could simply upload new files into the rawdata container and run the pipeline to generate clean outputs. This process saves time, reduces the chance of manual errors, and ensures consistency across datasets.

However, the system is not without limitations. The pipeline is most effective when incoming files follow a consistent structure. If future datasets contain different column names, formats, or file types, the existing pipeline would need to be modified or rebuilt. Although Azure Data Factory allows for more complex workflows, adapting the system to handle varied data structures would require more development time and planning.

There is also financial consideration. Azure Blob Storage and Data Factory are not free beyond the trial tier. Businesses must evaluate whether the benefits of automation and accuracy outweigh the costs of running these services in the long term. For organizations that handle frequent and consistent data imports, the value is clear. For others with smaller or more varied datasets, a more flexible or cost-effective solution might be more appropriate.

Conclusion: This project demonstrated how cloud tools like Azure Blob Storage and Azure Data Factory can be used to streamline a common data management challenge. Automating the process of cleaning raw data files not only improves efficiency but also enhances data quality and reliability. While there are some constraints, especially around cost and data variability, the overall approach offers a practical solution to a problem that many businesses face. With further customization, the pipeline could be scaled or adapted for broader applications in real-world environments.

Project Contributors:

Grace Akinyi
Michele Gipson
Briana Harper
Katie Lucey